

Распределение вычислительной нагрузки в многопроцессорной вычислительной системе

1 Постановка задачи

Имеется многопроцессорная вычислительная система (ВС) с одинаковыми узлами-процессорами, соединенными сетью. Задан набор программ, которые должны выполняться на этой ВС. Каждая программа выполняется на одном процессоре; допустимо выполнение на одном и том же процессоре нескольких программ. Между некоторыми программами происходит обмен данными с фиксированной скоростью. Такая архитектура используется в управляющих ВС, например, в системах авионики или автоматизации производства.



Каждая программа создает фиксированную нагрузку на процессор. Нагрузки на один и тот же процессор от разных программ суммируются. Суммарная нагрузка на процессор от всех выполняющихся на нем программ не должна превышать 100%.

Программы, выполняющиеся на разных процессорах, обмениваются данными через сеть; при этом создается нагрузка на сеть, измеряемая в Кбайт/с. Программы, выполняющимися на одном и том же процессоре, обмениваются данными через локальную память процессора; такой обмен не создает нагрузку на сеть. Нагрузки на сеть от разных пар взаимодействующих программ суммируются.

Чтобы избежать перегрузки сети, задается максимально допустимая нагрузка на сеть. В рамках этого ограничения, стараются распределить программы по процессорам так, чтобы минимизировать максимальную, по всем процессорам, нагрузку на процессор. Например: если есть два процессора и три программы, создающие нагрузку 10%, 30% и 50% соответственно, то распределение (10%+30%; 50%) лучше, чем (10%+50%; 30%), потому что в первом случае максимальная нагрузка на процессор равна 50%, а во втором она равна 60%.

Задано:

- число процессоров в ВС;
- число программ;
- для каждой программы: нагрузка на процессор (в %, не более 100);
- перечень пар программ, между которыми идет обмен данными;
- для каждой такой пары программ: интенсивность обмен данными (Кбайт/с);
- верхняя граница для нагрузки на сеть (Кбайт/с).

Требуется: распределить все программы по процессорам так, чтобы минимизировать максимальную нагрузку на процессор, и при этом не нарушить ограничение по нагрузке на сеть, а также не нагрузить ни один процессор более чем на 100%.

Решение задачи представляется в виде вектора $\bar{x}$ (его размер равен числу программ), в котором для каждой программы указан номер процессора, на который она распределена.

Для упрощения алгоритмов решения задачи и формирования наборов входных данных для исследования алгоритмов, на входные данные накладываются следующие дополнительные ограничения:

- создаваемая программой нагрузка на процессор может принимать значения 4, 11, 14, 21 (%);
- интенсивность обмена данными между двумя программами может быть равной 0 (нет обмена), 21, 49, 71, 99 (Кбайт/с);
- верхняя граница для нагрузки на сеть кратна 100 Кбайт/с.

2 Алгоритм случайного поиска с сужением области для распределения программ по процессорам

Данный алгоритм итеративно случайным образом генерирует решение задачи $\bar{x}$ и проверяет, лучше (меньше) ли оно по значению целевой функции $F(\bar{x})$ (т.е. максимальной, по всем процессорам, нагрузки на процессор), чем наилучшее из ранее найденных корректных (удовлетворяющих ограничениям) решений. Если лучше, то проверяется корректность сгенерированного решения; в случае корректности, это решение становится новым наилучшим.

Схема выполнения базового алгоритма следующая:

- 1) установить текущее наилучшее значение целевой функции (обозначим его за F_{best}) в 100;
- 2) случайным образом сформировать решение задачи (вектор $\bar{x}$);
- 3) если $F(\bar{x}) \geq F_{best}$, перейти к шагу 2 (сформированное решение не лучше, чем ранее найденное)
- 4) если $\bar{x}$ – некорректное решение (нарушает ограничения по нагрузке на сеть), перейти к шагу 2;

// на шаге 5 имеем корректное решение, лучшее чем ранее найденное

5) обновить текущее наилучшее решение и текущее наилучшее значение целевой функции:

$$\bar{x}_{best} := \bar{x}; F_{best} := F(\bar{x});$$

6) перейти к шагу 2.

Завершение алгоритма происходит, когда за 5000 попыток подряд не удалось улучшить значение F_{best} , т.е. сформировать корректное решение $\bar{x}$ такое, что $F(\bar{x}) < F_{best}$. В случае если при выполнении алгоритма не удалось найти ни одного корректного решения, результатом работы алгоритма должен быть массив, содержащий только значения (-1).

Не забывайте переинициализировать генератор случайных чисел, иначе при каждом прогоне он будет выдавать одно и то же!

Параллельный алгоритм: алгоритм работает в виде нескольких параллельно работающих «потоков», каждый из которых выполняет те же действия, что и базовый алгоритм. Потоки обновляют единое (общее для них) наилучшее решение и наилучшее значение целевой функции.

3 Формат входных данных

Набор входных данных (см. пункт «Задано» постановки задачи) задается в файле. Формат файла должен быть основан на XML. Конкретную структуру файла (элементы XML, их атрибуты) необходимо разработать.

4 Формат выходных данных

В стандартный вывод необходимо вывести:

В случае успеха:

- 1) строку «SOLVED»;
- 2) число итераций алгоритма;
- 3) элементы найденного решения $\bar{x}_{best}$, с разделителем-пробелом;
- 4) значение целевой функции на найденном решении.

Текст по каждому из пунктов 1 – 4 выводить на отдельной строке.

В случае неуспеха:

- 1) строку «FAILED»;
- 2) число итераций алгоритма.

5 Задания и система оценки

Основное задание (обязательное, оценивается в 5 баллов):

- 1) реализовать базовый алгоритм решения задачи в виде программы на языке C++ или Python;
- 2) проверить его работу на наборах данных с 4, 8, 16 процессорами; средним числом программ на процессор (число_программ / число_процессоров) не менее 6; средней

загрузкой процессора не менее 40%; каждая программа ведет обмен данными не менее чем с двумя другими программами; в каждом наборе входных данных должны присутствовать все указанные в Постановке задачи варианты загрузки процессора и интенсивности обмена данными;

(наборы данных должны быть присланы вместе с программой)

- 3) написать инструкцию к программе:
 - как собирать и запускать программу;
 - описание формата входного файла.

Дополнительное задание (не обязательное, оценивается в 5 баллов):

- 1) реализовать параллельный алгоритм в виде многопоточной или многопроцессной программы на том же языке, что и базовый алгоритм;
Примечание: в написании многопоточных/многопроцессных программ на Python есть важные тонкости; важно, чтобы ваша реализация выполнялась *действительно* параллельно, а не псевдопараллельно (в терминах Python - асинхронно);
- 2) исследовать, на сколько (в % относительно варианта с одним потоком/процессом) уменьшается время выполнения вашей программы при увеличении числа потоков/процессов (на наборах данных с 16 процессорами в ВС);
- 3) оформить результат исследования в виде диаграммы.

Поскольку алгоритм является рандомизированным, нужно выполнять 10 прогонов алгоритма на одном и том же наборе входных данных, с одним и тем же числом потоков/процессов, и время выполнения усреднять. Сравнивать следует усредненные времена выполнения для 1, 2, 4, 8 потоков/процессов.

Баллы за основное и дополнительное задания суммируются. Если при проверке задания выявлен плагиат, оценка за все части задания обнуляется; проверяющие *не проводят* анализ, чье задание является «первоисточником», а чье – «заимствованием».

6 Требования к программной реализации

- 1) программа должна быть реализована на языке C++ или Python;
- 2) инструкции должны быть представлены в формате .txt или .pdf;
- 3) помимо технической информации, инструкция должна содержать фамилию, имя и номер группы студента;
- 4) программа должна работать в среде ОС Linux (проверка будет проводиться в ОС Ubuntu 24.04), для работы с Ubuntu 24.04 можно использовать технологию WSL (Windows support for Linux¹) под ОС Windows 10 или 11;

¹ На самом деле – Linux support for Windows :)

- 5) текст программы должен быть хорошо читаемым, структурированным, и содержать комментарии в количестве, необходимом для его понимания;
- 6) программа должна диагностировать ситуации некорректного пользовательского ввода и содержимого входных файлов;
- 7) исходный код программы не должен выкладываться в публичные репозитории (github и т.п.)

7 Отправка работ

- 1) результаты выполнения заданий отправляются на адрес: asvk-nabor@asvk.cs.msu.ru
- 2) тема письма должна иметь вид: [номер группы] – MVS – Фамилия Имя, например: [211] – MVS – Самородов Антоний
- 3) материалы основного и дополнительного (при наличии) заданий должны быть оформлены в виде отдельных архивов формата .tar.bz2